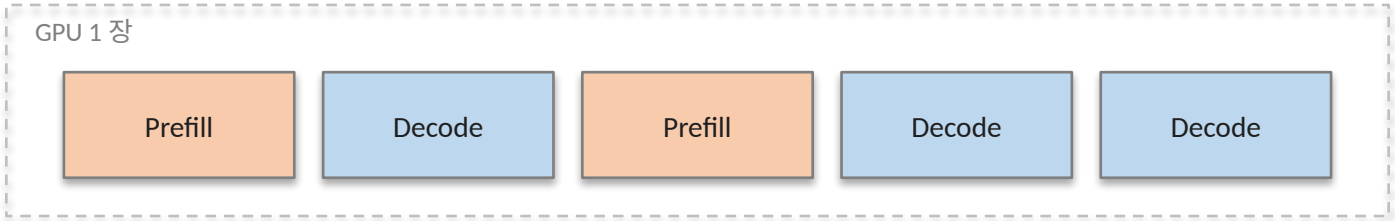
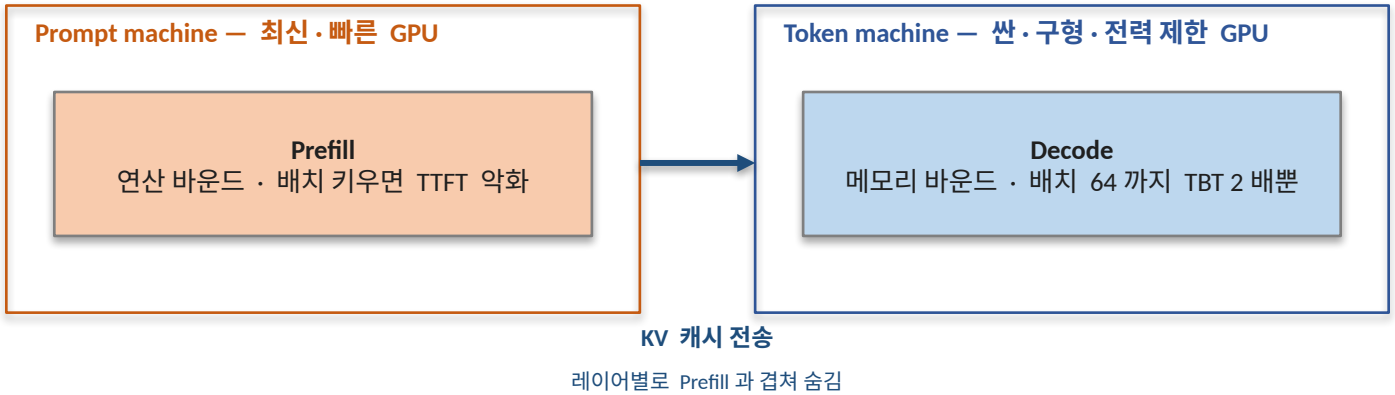
이기종 기법 ① LLM — Prefill / Decode 분할

기존 : 한 GPU 가 두 단계를 번갈아 처리 (혼합 배칭)



긴 프롬프트의 Prefill 이 끼어들면 그 스텝의 모든 Decode 가 느려짐 (TBT 튐) · Decode 만 돌 때는 연산기가 놂

Splitwise: 단계마다 전용 기계



■ 연산 바운드 ■ 메모리 바운드

우리 박스 (4090 + 3090) 에서 확인한 것 :
입력이 길고 출력이 짧은 요청 (code) 에서 4090 Prefill + 3090 Decode 가 통하고 , 출력이 길면 Decode 쪽 KV 메모리가 먼저 찬다 .

무엇을 나누나

- Prefill(프롬프트 처리) 과 Decode(토큰 생성) 를 다른 기계에
  - 요청은 Prompt 기계 → KV 전송 → Token 기계 순서

왜 이득인가

- Prefill 은 연산 집약 , Decode 는 메모리 대역폭에 묶여 최신 GPU 의 연산 · 전력을 못 씀
  - 섞어 돌리면 서로 간섭 → 나누면 각 단계에 맞는 배치 · 하드웨어 선택 가능

대가

- 요청마다 KV 캐시 (8B 모델 2K 프롬프트 ≈ 270 MB) 전송
- Token 기계에 KV 가 몰림 → 출력이 길면 메모리 용량이 먼저 한계

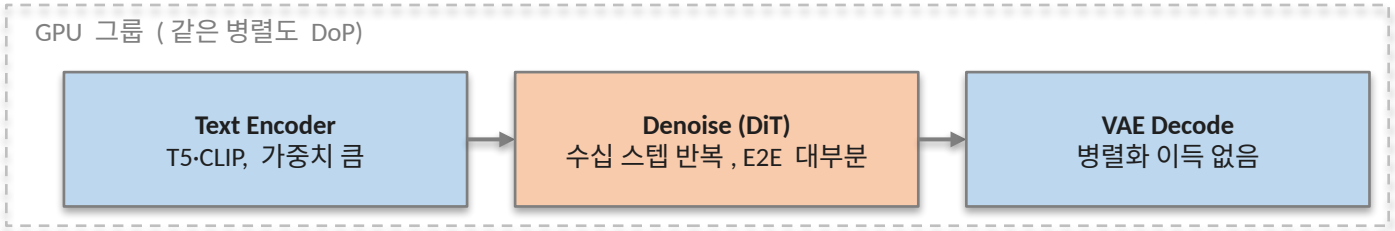
보고된 효과

- 같은 비용에서 처리량 1.4 배 (비용 20% 절감), 같은 전력 · 비용에서 2.35 배
- Decode 를 A100(구형) 에 둬도 처리량 유지

Patel et al., "Splitwise: Efficient Generative LLM Inference Using Phase Splitting", ISCA 2024

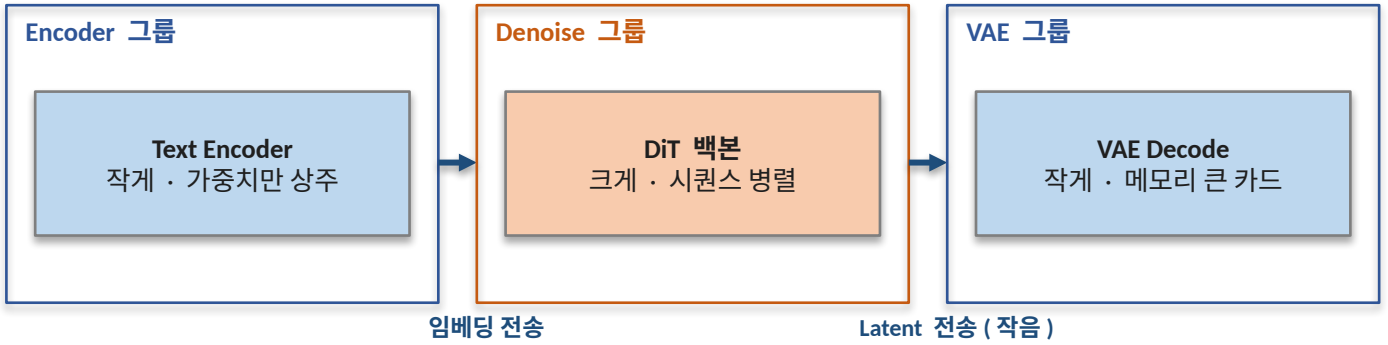
이기종 기법 ② DiT — Text Encoder / Denoise / VAE 분리

기존 : 한 GPU 그룹이 세 단계를 같은 병렬도로 통째로 처리



GPU 를 늘려도 VAE 시간은 안 줄어 전체에서 VAE 비중만 커짐 · 단계마다 필요한 GPU 수가 다른데 하나로 묶임

DDiT / DisagFusion: 단계별 독립 GPU 그룹 , 그룹마다 GPU 수 · 병렬도를 따로



■ 연산 바운드 ■ 메모리 바운드

우리 측정 (이전 세션):
Denoise 가 E2E 지연의 약 90%, VAE 가 메모리의 약 50% 를 차지 → 시간은 Denoise 에 , 메모리는 VAE 에 몰려 있어 같은 카드에 두면 둘 다 낭비 .

무엇을 나누나

- Text Encoder / DiT Denoise / VAE 를 독립 GPU 그룹에
  - 그룹마다 인스턴스 수와 병렬도 (DoP) 를 따로 정함

왜 이득인가

- Denoise 는 GPU 를 늘릴수록 빨라지지만 VAE 는 그렇지 않음 → 같은 병렬도로 묶으면 VAE 쪽 GPU 가 놂
- Encoder·VAE 는 가중치 · 활성 메모리가 커서 메모리 큰 카드가 , Denoise 는 연산 빠른 카드가 맞음

대가

- 단계 사이 임베딩 ·latent 전송 (latent 는 압축 공간이라 작음)
- 단계별 큐가 생겨 스케줄링이 필요

보고된 효과

- DDiT: p99 지연 최대 1.44 배 , 평균 30% 개선 (OpenSora, T5 + DiT + VAE)
- DisagFusion: 소비자 GPU(24 GB) 에서도 대형 모델 서빙 , 단계별 탄력 스케줄링

Huang et al., "DDiT: Dynamic Resource Allocation for Diffusion Transformer Model Serving", arXiv 2506.13497 (2025) · Zha et al., "DisagFusion", arXiv 2605.25550 (2026)

이기종 기법 ③ VLM — Encode / Prefill / Decode 분할

기존 : 인코더와 LLM 이 같은 GPU 에 함께 상주



인코딩이 Prefill 을 막아 TTFT 증가 · 두 모델이 메모리를 나눠 써서 배치와 이미지 수가 제한됨

EPD Disaggregation: 세 단계를 전용 자원에



■ 연산 바운드 ■ 메모리 바운드

LLM PD 분할과의 관계 :
PD 분할 앞에 Encode 단계가 하나 더 붙은 꼴 . 인코더는 LLM 가중치가 필요 없어 작은 · 구형 카드에 따로 둘 수 있다 .

무엇을 나누나

- Encode(이미지 · 영상 → 토큰) / Prefill / Decode 를 전용 자원에
- 단계마다 배칭 · 병렬화 · 스케줄링을 따로 설계

왜 이득인가

- 인코딩은 연산 집약이고 토큰 수를 크게 늘려 Prefill 을 지연시킴
- 인코더와 LLM 을 한 GPU 에 두면 메모리를 나눠 써서 배치 · 이미지 수가 제한됨

대가

- 멀티모달 토큰과 KV 캐시 전송 (토큰은 캐시해 재사용 가능)
- 단계별 자원 배분을 워크로드에 맞춰 정해야 함

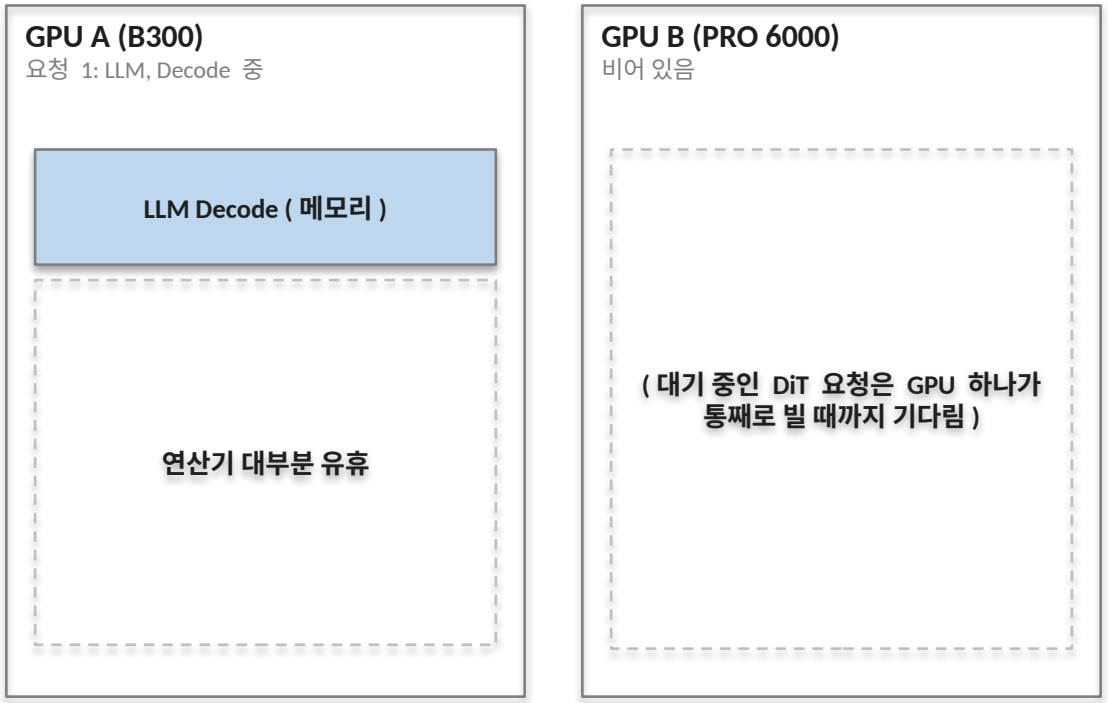
보고된 효과

- 메모리 15 배 절감 , 배치 22 배 , 요청당 이미지 10 배
- SLO 만족률 최대 90~100% 개선 , TTFT 최대 71% 감소

Singh et al., "Efficiently Serving Large Multimodal Models Using EPD Disaggregation", ICML 2025

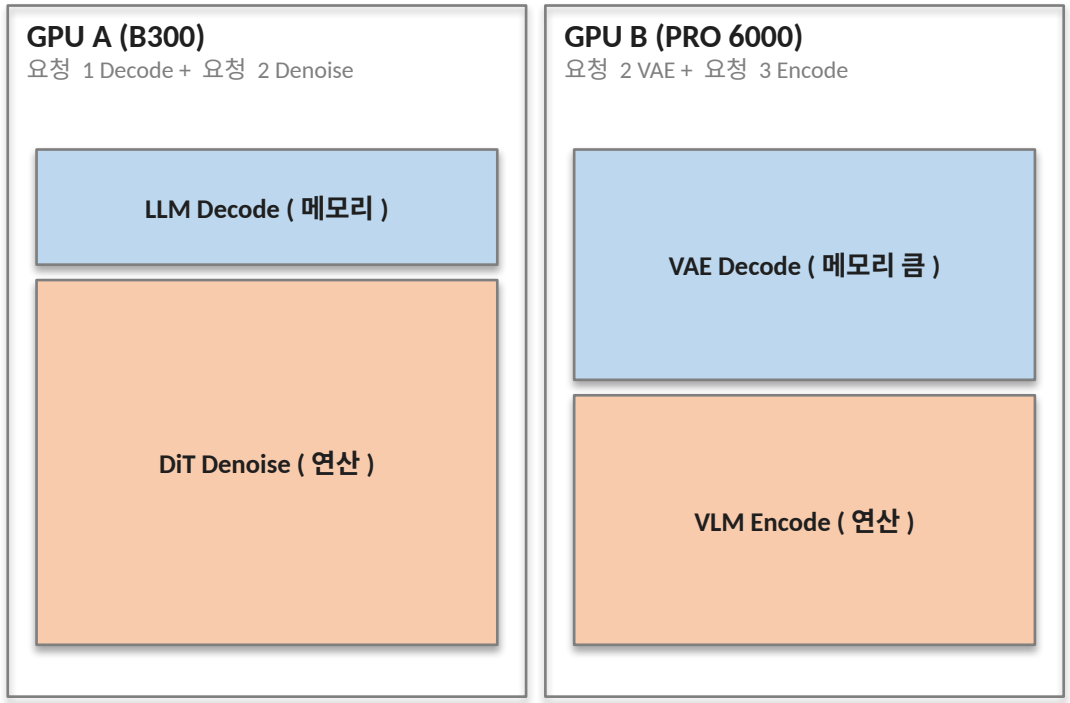
우리가 하려는 것 — 비어 있는 자원에 ' 맞는 단계 ' 를 넣는다

지금 : 요청 단위로 통째로 배정



한 장은 메모리만 쓰며 연산기가 놀고 , 한 장은 통째로 비어 있는데도 다음 요청은 기다린다

우리 : 분할 기법으로 단계 단위로 채운다



요청을 단계로 쪼개 특성이 보완되는 곳에 넣으면 같은 두 장으로 세 요청을 동시에 처리

Solver 가 정하는 것 : 언제 , 어느 요청의 어느 단계를 , 어느 카드에 넣어야 SLO 를 지키면서 비용 효율이 최대인가 — 분할이 늘 이득은 아니므로 (전송 · 간섭 · 메모리) 모델로 미리 판단한다